1. What am I making?

RADIX ProofPath helps a candidate understand job readiness. Five teammates own five parts: JD Analytics -> Resume Parsing -> Profile Builder -> Talent Check -> Skill Matching. You own part 4.

Compare twelve self-ratings with one company's bar.

Show a candidate where their own stated levels meet or fall below a selected company's twelve expected levels. Explain every row and the overall calculation.

Why your part matters

Company readiness is different from matching a single advert. The user needs to see which broad categories need attention and where the standards came from.

What the user gives you

A saved profile from Module 3 and a selected company with an authorized twelve-category benchmark.

What you return

An overall readiness percentage when all ratings are known, twelve comparison rows, gaps, actions and the benchmark source. Missing ratings produce an incomplete result, not fake zero ability.

A small example

In our invented lab, Asha rates SQL as 4 and Synthetic Data Co expects 9. SQL attainment is 4/9; its share is (4/9)/12 x 100 = 3.7037 points. All twelve lab ratings at 4 give 74.9% overall.

Start with the working code

This is a working starting point, not an instruction to rebuild an app. Clone the WHOLE repository so shared support is available. Work mainly in talent_check/. Read this one guide, run it, then improve a problem you can demonstrate.

Your folder has this guide as PDF and Markdown, backend/, frontend/, tests/, samples/ and run.ps1. Shared support is stored once at the root. The Markdown contains the same teaching content and full examples as this PDF.

2. What exactly must I build?

Organizer requirements - keep these working

- Load a saved candidate profile and let the user select one company.
- Compare across all twelve official RADIX categories, showing candidate and required levels and the gap.
- Return a readiness score and per-category breakdown using an explicitly defined formula. The brief permits a team scoring decision; it does not prescribe this formula.
- Use real organizer expectations for official readiness. The current organizer table is staging_company_skill_levels; access stays read-only. Run independently using clearly synthetic fixtures while access is missing.

Already implemented

Read-only benchmark loading/mapping/validation, explicit synthetic mode, twelve-row scoring, unknown-rating handling, source receipts and the comparison screen already work. Career GPS reuses this same scorer.

Your development checklist

- 1. Run the working screen with Synthetic Data Co. Independently calculate two rows and the whole percentage from the provided values.
- 2. Compare two candidate profiles or change one explicit rating. Show a real arithmetic change, not a hardcoded score.
- 3. Test Unknown, missing company, malformed benchmarks and unavailable real source. Improve a reproduced failure and add a test.
- 4. When the Scrum Master has authorized organizer access, confirm actual column mappings and all twelve values. Keep real-source acceptance pending until that succeeds.

Currently missing or unverified

Actual organizer schema, authorized access and real company values are unavailable. Live Supabase acceptance is pending. Never fill these gaps with invented Google/Microsoft/Oracle expectations.

Extra ProofPath features

ProofPath shows row contributions, source provenance and useful next actions. Evidence coverage is a separate count; adding quotes does not change readiness unless the user changes a self-rating.

3. How does it work?

1. Choose profile	**2. Choose source**	**3. Validate company**
4. Compare rows	**5. Calculate**	**6. Explain**

- 1. Choose profile: Read the saved candidate's twelve explicit self-ratings. Skill mentions do not supply those numbers.
- 2. Choose source: The user explicitly selects synthetic mode for lab data. Real mode requires a genuine authorized source.
- 3. Validate company: Require exactly twelve unique category values in range 1-10. Missing or invalid data returns a useful error.
- 4. Compare rows: For each category: attainment = min(candidate / required, 1). A higher self-rating cannot contribute more than that category's full share.
- 5. Calculate: If all twelve ratings are known, average the twelve attainments x 100, rounded to one decimal. Otherwise show a null overall score and unknown rows.
- 6. Explain: Show candidate, required, shortfall, contribution and source. Save the result receipt; pass the same evaluator to GPS.

Words you will see

Backend = Python server code. Frontend = React browser screen. API = an address the screen calls to send/receive data. A contract/schema is the agreed field list, defined in shared/contracts/models.py. An LLM is a language model that interprets text; Groq runs it. Supabase is the cloud database service.

The shared RADIX categories

COD Coding; DSA Data Structures & Algorithms; OOD Object-Oriented Design; APTI Aptitude; COMM Communication; AI Artificial Intelligence; CLOUD Cloud; SQL SQL; SWE Software Engineering; SYSD System Design; NETW Networking; OS Operating Systems.

Each skill has exactly one category, or OTHER. Descriptive skill families are extra UI filters, not new rated categories. Unknown competency is null, never zero. Confidence describes extraction support, not ability. What-if changes never save a real profile.

4. Inputs and outputs

Module 3 supplies the saved Profile. The user sees readiness; Module 5's GPS reuses the pure calculation for previews.

POST /api/talent-check/evaluate

Send this complete JSON request. profile_id must identify a profile already saved in your own workspace. This example uses the synthetic candidate.json with twelve explicit level-4 ratings and company.json's invented lab values.

Complete input example: talent_check/samples/input.json

```
{
  "profile_id": "84027300-d488-4f39-a1ab-c1e6f8573e61",
  "company_id": "lab-data",
  "benchmark_mode": "synthetic"
}
```

Important fields

- profile_id / company_id / benchmark_mode: profile_id and a company selection are required for this route. Mode defaults to real; synthetic must be chosen explicitly. Do not silently switch modes on failure.
- skillset_gap: Exactly twelve rows. required_level is a benchmark, candidate_level is a self-rating, gap is true/false/null and contribution is that row's score share.
- readiness_score / status: 0-100 only when all twelve ratings exist; otherwise null and incomplete. rating_completeness counts known ratings, not proven ability.
- benchmark_source / hash / receipt_id: Where the values came from, which snapshot was used and the saved result reference. Synthetic source notice must remain visible.

GET /api/talent-check/companies?mode=synthetic lists lab companies. Without that parameter it requests real data. GET /api/talent-check/benchmark-status explains real-source availability. Evaluations use the profile's current saved revision and record the benchmark snapshot. Unknown is never converted to zero.

The next page(s) show the COMPLETE captured response from the existing API, including defaults and empty fields. Long lines wrap for reading. Use samples/output.json for exact machine-readable JSON. IDs, receipt IDs, source hashes and times can differ between runs; do not hardcode them. JSON null means unknown/not present; [] means an empty list.

Complete output example

talent_check/samples/output.json


```
{
  "company": "Synthetic Data Co",
  "company_id": "lab-data",
  "profile_id": "84027300-d488-4f39-a1ab-c1e6f8573e61",
  "profile_revision": 1,
  "skillset_gap": [
    {"category_code": "COD", "category_name": "Coding", "required_level": 7.0,
  "candidate_level": 4.0, "gap": true, "shortfall": 3.0, "status": "below_expected",
  "attainment": 0.5714285714285714, "contribution": 4.7619, "evidence": [],
  "rating_origin": "self_reported", "recommended_action": "Explore improving Coding from
  4 toward 7"},
    {"category_code": "DSA", "category_name": "Data Structures & Algorithms",
  "required_level": 5.0, "candidate_level": 4.0, "gap": true, "shortfall": 1.0, "status":
   "below_expected", "attainment": 0.8, "contribution": 6.6667, "evidence": [],
  "rating_origin": "self_reported", "recommended_action": "Explore improving Data
  Structures & Algorithms from 4 toward 5"},
    {"category_code": "OOD", "category_name": "Object-Oriented Design", "required_level":
   4.0, "candidate_level": 4.0, "gap": false, "shortfall": 0.0, "status":
  "meets_expected", "attainment": 1.0, "contribution": 8.3333, "evidence": [],
  "rating_origin": "self_reported", "recommended_action": "Maintain capability and add
  supporting evidence"},
    {"category_code": "APTI", "category_name": "Aptitude", "required_level": 8.0,
  "candidate_level": 4.0, "gap": true, "shortfall": 4.0, "status": "below_expected",
  "attainment": 0.5, "contribution": 4.1667, "evidence": [], "rating_origin":
  "self_reported", "recommended_action": "Explore improving Aptitude from 4 toward 8"},
    {"category_code": "COMM", "category_name": "Communication", "required_level": 7.0,
  "candidate_level": 4.0, "gap": true, "shortfall": 3.0, "status": "below_expected",
  "attainment": 0.5714285714285714, "contribution": 4.7619, "evidence": [],
  "rating_origin": "self_reported", "recommended_action": "Explore improving
  Communication from 4 toward 7"},
    {"category_code": "AI", "category_name": "AI", "required_level": 8.0,
  "candidate_level": 4.0, "gap": true, "shortfall": 4.0, "status": "below_expected",
  "attainment": 0.5, "contribution": 4.1667, "evidence": [], "rating_origin":
  "self_reported", "recommended_action": "Explore improving AI from 4 toward 8"},
    {"category_code": "CLOUD", "category_name": "Cloud", "required_level": 5.0,
  "candidate_level": 4.0, "gap": true, "shortfall": 1.0, "status": "below_expected",
  "attainment": 0.8, "contribution": 6.6667, "evidence": [], "rating_origin":
  "self_reported", "recommended_action": "Explore improving Cloud from 4 toward 5"},
    {"category_code": "SQL", "category_name": "SQL", "required_level": 9.0,
  "candidate_level": 4.0, "gap": true, "shortfall": 5.0, "status": "below_expected",
  "attainment": 0.4444444444444444, "contribution": 3.7037, "evidence": [],
  "rating_origin": "self_reported", "recommended_action": "Explore improving SQL from 4
  toward 9"},
    {"category_code": "SWE", "category_name": "Software Engineering", "required_level":
  5.0, "candidate_level": 4.0, "gap": true, "shortfall": 1.0, "status": "below_expected",
   "attainment": 0.8, "contribution": 6.6667, "evidence": [], "rating_origin":
  "self_reported", "recommended_action": "Explore improving Software Engineering from 4
  toward 5"},
```

Complete output - continued


```
    {"category_code": "SYSD", "category_name": "System Design", "required_level": 4.0,
  "candidate_level": 4.0, "gap": false, "shortfall": 0.0, "status": "meets_expected",
  "attainment": 1.0, "contribution": 8.3333, "evidence": [], "rating_origin":
  "self_reported", "recommended_action": "Maintain capability and add supporting
  evidence"},
    {"category_code": "NETW", "category_name": "Networking", "required_level": 3.0,
  "candidate_level": 4.0, "gap": false, "shortfall": 0.0, "status": "above_expected",
  "attainment": 1.0, "contribution": 8.3333, "evidence": [], "rating_origin":
  "self_reported", "recommended_action": "Maintain capability and add supporting
  evidence"},
    {"category_code": "OS", "category_name": "Operating Systems", "required_level": 4.0,
  "candidate_level": 4.0, "gap": false, "shortfall": 0.0, "status": "meets_expected",
  "attainment": 1.0, "contribution": 8.3333, "evidence": [], "rating_origin":
  "self_reported", "recommended_action": "Maintain capability and add supporting
  evidence"}
  ],
  "readiness_score": 74.9,
  "status": "complete",
  "rating_completeness": 12,
  "evidence_coverage": 0,
  "rules_version": "readiness-1.0-equal-capped",
  "benchmark_source": {"source_mode": "synthetic_test", "source_table":
  "synthetic_fixture_not_RADIX", "fetched_at": null, "content_hash":
  "c465f2868ffe4ba8db4bfbd23d7000a6064e318c9887809e0fc0f2a1fd0c3aa5", "notice":
  "SYNTHETIC LAB DATA. Invented companies and levels for testing only; not official RADIX
   expectations."},
  "benchmark_hash": "c465f2868ffe4ba8db4bfbd23d7000a6064e318c9887809e0fc0f2a1fd0c3aa5",
  "methodology": "Mean of capped candidate/required attainment across all 12 categories.
  Self-reported levels; evidence links do not verify ability. Not an employer decision or
   hiring probability.",
  "receipt_id": "ceb7b6a8-5844-485b-9094-61b835e2e836"
}
```

5. How do I work on the code?

Install and run - Windows PowerShell

Install Python 3.10+, Node.js 24 LTS (includes npm), Git and an editor such as VS Code. Accept the installers' PATH option, reopen PowerShell and clone the full GitHub repository. Open a terminal at the repository root - the folder containing package.json.

```
.\talent_check\run.ps1 setup
# Terminal 1:
.\talent_check\run.ps1 api
# A SECOND terminal, same root:
.\talent_check\run.ps1 web
```

Open http://127.0.0.1:5173/. Server API help is at http://127.0.0.1:8000/docs. setup creates .venv and installs locked Python/npm dependencies plus Chromium for browser tests. Run setup once; no key is needed. Keep api and web running; Ctrl+C stops your own server. Only one module uses 8000/5173 at a time.

Open these files first

- talent_check/backend/scoring.py - Open first. Short, deterministic twelve-category calculation; used again by GPS.
- talent_check/backend/benchmark_repository.py - Loads and validates authorized real data or explicitly chosen synthetic values.
- talent_check/backend/router.py - Loads the saved profile/company, runs scoring and records the result.
- talent_check/frontend/TalentCheckPage.tsx - Shows the score, source information, twelve rows and related evidence.

shared/contracts/models.py defines Profile, ReadinessRow and ReadinessResult. shared/integrations/store.py loads profiles; receipts.py records source snapshots. Do not replace the shared model or add a second, conflicting GPS formula.

Make one useful change

Read a failing example, locate the responsible function, change that part and add a test in your tests/ or frontend test file. Do not rename shared fields, duplicate common code or rewrite other modules. Ask the Scrum Master before changing shared contracts, storage or provider code.

What should I see?

Expect company/source selection, Run Talent Check, overall score or Incomplete, twelve readable rows, candidate-versus-required values and source details. The synthetic label must be obvious.

Load synthetic inputs supplies pre-reviewed Mira records for independent operation. It is not fresh extraction. The tiny Asha terminal examples and the richer Mira browser examples are intentionally different inputs.

6. How do I know it works?

```
.\talent_check\run.ps1 example
.\talent_check\run.ps1 test
```

example runs the small teaching case in a fresh temporary database and checks its result. test runs your real standalone API check, Python tests, React tests, type checking, lint, your production build and your browser test. It stops on failure. Both use synthetic data; tests force rules mode and do not prove live LLM quality.

- Known arithmetic: Use samples/candidate.json and company.json. The example command checks all twelve level-4 ratings give 74.9. Browser: Load synthetic inputs, choose Synthetic Data Co and run.
- One category: SQL is 4 versus 9: shortfall 5, attainment 0.4444..., contribution 3.7037 points. Wrong: confidence changes this number or evidence creates a higher level.
- Unknown rating: Clear a saved rating in the integrated Profile Builder, then assess. Expect twelve rows but incomplete/null score. The independent example's candidate is separate from your browser's records.
- Unavailable source: Choose missing/real company data without credentials. Expect a useful source error; do not display lab values as official or a fake 0% readiness.
- Bad data / repeat: Tests reject duplicate/missing/out-of-range benchmark categories. Repeat the same inputs: arithmetic must be identical even if receipt IDs or fetch timestamps differ.

Use an unfamiliar document too

Use only documents you own or have permission to process. Before running extraction, list expected skills and supporting words yourself. Compare missing and invented skills, sections, groups and quotes. A successful API response or valid JSON is not proof of correct extraction. Do not commit real CVs or send them to Groq without explicit permission.

If something fails

Read the FIRST error. Missing Python/npm: install and reopen the terminal. Missing packages: rerun setup. Browser cannot reach API: keep api running. Port busy: stop your own earlier server or coordinate a free port; do not kill unknown processes. Wrong extraction: compare the source, review and add a regression case.

7. Missing resources and what to do

What you can do now

All local teaching inputs are synthetic. No organizer pack, API key or cloud account is needed to run, edit and test your module. The tiny samples teach fields and behavior; the larger reconstructed pack remains under data/synthetic/. Neither is official organizer data.

Your samples - every file has a purpose

- candidate.json - Full synthetic candidate with twelve deliberately invented ratings of 4.
- company.json - One labelled invented company and all twelve required levels.
- input.json - Complete upload description or JSON request shown in this guide.
- output.json - Complete response captured from that actual API request.

When organizer access arrives

The Scrum Master needs the authorized Supabase project URL, read permission/credential, actual company/category/level columns and any required company IDs or filters for staging_company_skill_levels. Configure private root .env using RADIX_BENCHMARK_URL, RADIX_BENCHMARK_KEY (or authorized access token) and RADIX_BENCHMARK_MAPPING. An authorized local export can use RADIX_BENCHMARK_EXPORT instead. Do not guess a schema or invent expected levels.

Verify twelve unique valid values per company and test real mode. Until then show source unavailable and use explicitly selected synthetic mode for practice. Never write the organizer staging table; never publish its keys or private exports.

8. Submit and explain your module

Completion checklist

[ ] I can run/explain it. [ ] Required inputs/outputs work. [ ] Normal/failure cases checked. [ ] Tests passed or failures reported. [ ] Contracts/source links preserved. [ ] No secrets/private CVs committed.

Push to YOUR own GitHub

Create an EMPTY private repository on your own account. From your full working clone, replace the URL below with YOUR new repository, review/stage only your changes, then run:

```
git remote rename origin scrum-master
git remote add origin https://github.com/YOU/YOUR-REPO.git
git add talent_check
.\.venv\Scripts\python.exe scripts/scan_handoff.py --staged
git commit -m "Verify my RADIX module"
git push -u origin main
```

Keep the whole working clone, including your source/tests/samples, guides, launcher and shared support. Never force-push. Grant the Scrum Master access if private.

Return URL, git rev-parse HEAD, changed files, actual test commands/results, three normal + three failure cases, synthetic screenshots and limitations. Distinguish implemented, mock-tested, live-verified and pending. The Scrum Master handles integration; you need not solve central merges.

A 45-second explanation for judges

I compare a saved candidate's explicit self-ratings with one company's twelve-category expectations. Every row shows the required level, candidate level, gap and contribution. We average capped candidate-to-required ratios, so the calculation is explainable and repeatable. Unknown ratings keep the overall result incomplete. Lab values are visibly synthetic; official readiness needs authorized organizer data. Adding evidence does not raise ability. Career GPS calls this same scorer, so its preview uses consistent mathematics without saving profile changes.

Three likely questions

- Is this a hiring probability? No. It is a transparent comparison of self-ratings and the chosen expectations.
- What if a rating or benchmark is missing? A missing rating gives incomplete/null readiness; missing or invalid benchmarks give a source error.
- Are the lab values official? No. They are invented and labelled; real organizer values still need authorized verification.

Copy this prompt into your AI assistant

Help me complete module 4: Talent Check step by step using this guide. First run it; request the exact section-5 source files and tests before editing. Preserve working code and shared fields. Fix a reproduced issue and test it. Keep confidence separate from competency and scenarios separate from saved profiles. Never invent official data or request pasted keys. Help me submit my own GitHub URL, tested commit, results and honest limitations.